Programa:	1st Startup Manager 1.0	
Protección:	ASProtect 2.0 [Registered] + Trial 30 días	
Objetivos:	Desempakar y evitar que espire	
URL descarga:	http://www.1stdesktop.com	
Dificultad:	No es tan difícil	
Herramientas:	OllyDbg v1.10 + ImpRec 1.6 + PeiD 0.93	
Cracker:	SyXe'05 ... [hAcK-c0d3d] ... [CracksLatinoS]	Fecha: 06-11-05

Hola a todos los CracksLatinos..!!

Hoy vamos a abordar una de las protecciones que están más de moda hoy en día, se trata del ASProtect 2.0 [Registered] una protección que es capaz de darnos problemas debido al ejército de secciones que crea en tiempo de descompresión, pero no es gran cosa cuando le metes mano donde se la tienes que meter ;-)

He de decir que en un inicio [ZekkT] desmontó esta protección con un injerto y un script, ya que como digo supo bien donde tocar, hoy vamos a ver de una forma más manual (y tediosa si me permitís) como desempakar un programa comprimido con esta versión de ASProtect, trabajaremos con el "1st Startup Manager 1.0" que viene además con un Trial de 30 días (creo, pues adelanté el reloj 1 año y expiró, jeje..)

Además de esto veremos algunos truquitos que hace para redireccionar las API's ya que es el fuerte de la protección, no entraremos en demasiado detalle pero sí que veremos algunas cosillas interesantes, divho esto procedemos al atake…

Como no nuestro amigo PEiD nos va a decir con qué está empaquetado en caso de estarlo:

ASProtect 2.0x Registered -> Alexey Solodovnikov

Pues ya lo estamos viendo lo que hemos dicho antes, abrimos un Olly y cargamos el programa, ocultamos el debugger con el plugin IsDebuggerPresent o bien ponemos un BP en dicha api y le cambiamos el byte de siempre (esto no es problema) una vez ocultado ya corre bien bajo Olly, lo primero que haremos será descubrir el OEP, utilizamos el método de las excepciones, llegamos hasta la última:

```
00C3E8F9   C601 9E        MOV BYTE PTR DS:[ECX],9E
00C3E8FC   BB 71FE8A67    MOV EBX,678AFE71
00C3E901   64:8F06        POP DWORD PTR FS:[ESI]
00C3E904   0000           ADD BYTE PTR DS:[EAX],AL
00C3E906   83C4 04        ADD ESP,4
```

Ahora ponemos el BPM en la primera sección y la superamos con Shift+F9 para caer en el buscado OEP, vemos que no hay stolen bytes:

```
00401240   55             PUSH EBP
00401241   89E5           MOV EBP,ESP
00401243   83EC 08        SUB ESP,8
00401246   C70424 0200000 MOV DWORD PTR SS:[ESP],2
0040124D   E8 B2EDB300    CALL 00F40004
00401252   4F             DEC EDI
00401253   E8 A8FEFFFF    CALL 1stStart.00401100
00401258   90             NOP
```

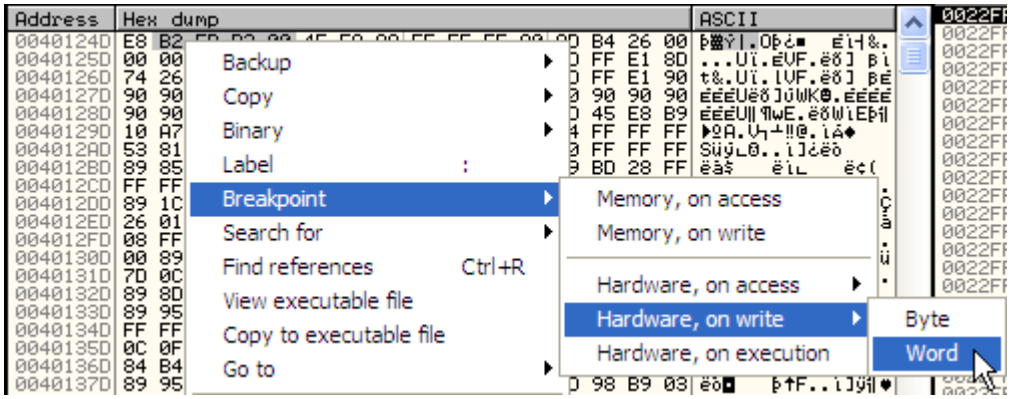
Venga ya tenemos el primer dato, lo anotamos como siempre hacemos:

OEP: 401240

Si nos fijamos bien en la imagen vemos un CALL que apunta a una zona de ASProtect:

0040124D E8 B2EDB300 CALL 00F40004

Este CALL nos va a ayudar bastante en nuestro trabajo ya que nos servirá de referencia, nos ponemos sobre él y hacemos un "Follow in dump->Selection" para ver los bytes de la instrucción en la ventana de datos, una vez allí le ponemos un "Break Point Hardware, on Write->Word" para que pare cuando se escriban los bytes finales que forman la instrucción del call, de esta manera pararemos en el momento en que se escriban estos bytes y podremos traceare la zona de código donde escribe estos calls que no nos gustan nada, ponemos el HBP como en la imagen:



notar como despreciamos el E8 que es el Opcode del salto, nosotros queremos seguir el rastro de los bytes siguientes al E8, una vez puesto el BP reiniciamos Olly, ahora vamos a tildar las excepciones para que no se detenga en ellas, tengo que decir que este ASProtect no borra los HBP's como sí hacía en versiones anteriores, mejor para nosotros, una vez configurado Olly para que no se detenga en las excepciones y estando parados en el EP damos RUN (F9) al poco para por escritura:

```
00C1C23F   8B7D F0      MOV EDI,DWORD PTR SS:[EBP-10]
00C1C242   881C3E       MOV BYTE PTR DS:[ESI+EDI],BL
00C1C245   FF45 F0      INC DWORD PTR SS:[EBP-10]
00C1C248   FF4D EC      DEC DWORD PTR SS:[EBP-14]
00C1C24B ^ 75 95        JNZ SHORT 00C1C1E2
00C1C24D   5F           POP EDI
```

pero no hay nada interesante, va escribiendo byte a byte pero no pone los bytes finales que antes vimos, que recordemos eran B2EDB300, o sea que volvemos a pulsar F9 para que corra y vuelve a parar aquí:

```
00C126B7   F3:A5        REP MOVS DWORD PTR ES:[EDI],DWORD PTR DS:[ESI]
00C126B9   89C1         MOV ECX,EAX
00C126BB   83E1 03      AND ECX,3
00C126BE   F3:A4        REP MOVS BYTE PTR ES:[EDI],BYTE PTR DS:[ESI]
00C126C0   5F           POP EDI
```

está copiando datos pero todavía no ha escrito los bytes que queremos, F9 otra vez:

```
00C3A816   45           INC EBP
00C3A817   8945 00      MOV DWORD PTR SS:[EBP],EAX   <- Guarda los bytes
00C3A81A   8B4424 10    MOV EAX,DWORD PTR SS:[ESP+10]
00C3A81E   8B00         MOV EAX,DWORD PTR DS:[EAX]
00C3A820   894424 14    MOV DWORD PTR SS:[ESP+14],EAX
00C3A824 v EB 0C        JMP SHORT 00C3A832
```

Si miramos en los registros vemos que EAX contiene el valor buscado:

```
Registers (FPU)
EAX 00B3EDB2   <- En EAX los bytes buscados..!!
ECX 0022FEF0
EDX 00F4000C
EBX 00C70788
ESP 0022FEF8
EBP 0040124E 1stStart.0040124E
ESI 00EC0E9D
EDI 00C707C8
```

Hemos localizado la zona donde redirige los CALL's a zonas del packer, es hora de tracear un poco, estamos en un bucle que empieza en :C3A6B2 y termina en :C3A857 dondehay un JA cuando este deja de saltar el loop termina y finaliza el trabajo de los CALL's. Voy a tratar de explicar un poco lo que hace ASProtect en este punto, esto surgió tras una hora larga de estudio frente a la pantalla, habia leído el tute de [ZekkT] donde desempaka un ASProtect de estos pero de distinta manera, entonces pensé ¿porqué no atavar como hizo Ricardo en sus tutes de Armadillo con destrucción de tabla? Recordaba que para el IAT Scrabmled seguía la pista a un CALL o JMP a la iat en busca del lugar donde escribía los bytes, a partir de ahí cambiaba la base para que entrara dentro del .exe esto sería similar salvo que no tenemos que cambiar ninguna base, ni si quiera sabía lo que me iba a encontrar pero me adentré en busca de no sé que ;-)) cuando descubrimos la zona en la que guarda los bytes finales (imagen anterior) sólo es cuestión de traceo y de paciencia tratando de entender lo que va haciendo para poder desmontarle el chiringuito, es evidente que contra mejor entendamos lo que hace mayores posibilidades tendremos de superar la protección. Aún no sé como explicar lo que fui viendo veremos como sale esto…

Como digo estamos en un bucle donde va tratando todos los CALL's que va a redirigir, entonces de alguna manera debe saber en qué dirección poner estos CALL y que todos sabemos reemplazan saltos y/o llamadas a la iat, bien entonces necesita además conocer

la api a la que debe de ir el/la salto/llamada a partir de aquí y sabiendo esto podemos continuar, veamos un poco el código con comentarios que nos ayuden a entenderlo mejor, si llegamos a lo que es el final del bucle:

```
00C3A839   8BC8          MOV ECX,EAX
00C3A83B   038B E4000000 ADD ECX,DWORD PTR DS:[EBX+E4]
00C3A841   8BD6          MOV EDX,ESI
00C3A843   8BC3          MOV EAX,EBX
00C3A845   E8 8EE5FFFF   CALL 00C38DD8
00C3A84A   FF0C24        DEC DWORD PTR SS:[ESP]
00C3A84D   03B3 E4000000 ADD ESI,DWORD PTR DS:[EBX+E4]
00C3A853   833C24 00     CMP DWORD PTR SS:[ESP],0
00C3A857 ^ 0F87 55FEFFFF JA 00C3A6B2
00C3A85D   53            PUSH EBX
00C3A85E   E8 5D000000   CALL 00C3A8C0
00C3A863   0183 EC000000 ADD DWORD PTR DS:[EBX+EC],EAX
00C3A869   B0 01         MOV AL,1
00C3A86B   83C4 24       ADD ESP,24
00C3A86E   5D            POP EBP
00C3A86F   5F            POP EDI
00C3A870   5E            POP ESI
00C3A871   5B            POP EBX
00C3A872   C3            RETN
```

y seguimos el JA mientras se cumpla la condición de salto (vemos que [ESP] se va decrementando de a 1 y empieza valiendo 2Eh o 2Fh) esto quiere decir que el bucle iterará 46 ó 47 veces lo que indica que debe de haber estos mismos CALL's a parchear, siguiendo el salto caemos en lo que será nuestro primer punto de apoyo:

```
00C3A6B2   33C0          XOR EAX,EAX                          ;el Ja viene aqui...
00C3A6B4   8A07          MOV AL,BYTE PTR DS:[EDI]
00C3A6B6   8D0440        LEA EAX,DWORD PTR DS:[EAX+EAX*2]
00C3A6B9   8B6C83 68     MOV EBP,DWORD PTR DS:[EBX+EAX*4+68]
00C3A6BD   8BC6          MOV EAX,ESI
00C3A6BF   FFD5          CALL EBP
00C3A6C1   8BE8          MOV EBP,EAX
00C3A6C3   036B 24       ADD EBP,DWORD PTR DS:[EBX+24]
00C3A6C6   03AB E0000000 ADD EBP,DWORD PTR DS:[EBX+E0]        ;en EBP la direccion del CALL...
00C3A6CC   EB 01         JMP SHORT 00C3A6CF
00C3A6CE   E8 33C08A47   CALL 484E6706
00C3A6D3   098D 04408B54 OR DWORD PTR SS:[EBP+548B4004],ECX
```

El salto nos pone en :C3A6B2 después hace un "CALL EBP" que nos devuelve un valor en EAX que después pasa a EBP, a este valor se le añade el imagebase 400000 y finalmente otro valor más para dejar en EBP la dirección del CALL, jejeje… veamos los registros:

```
Registers (FPU)
EAX 2AEE7E02
ECX 00000000
EDX 00EC0FB5
EBX 00C70788
ESP 0022FEF8
EBP 00420F7C 1stStart.00420F7C
ESI 00EC0FED
EDI 00C707C8
```

Dirección del CALL a redirigir..!!

Esta dirección nos va a ayudar bastante, ponemos un HBP en ella, para asegurarnos de que vamos bien abriremos otro Olly y llegaremos al OEP en este mismo programa y allí lo dejaremos parado para ir mirando, de momento vamos a :420F7C a ver si hay un CALL:

```
00420F71   894424 04     MOV DWORD PTR SS:[ESP+4],EAX
00420F75   C70424 0000000 MOV DWORD PTR SS:[ESP],0
00420F7C   E8 83F0BD00   CALL 01000004
00420F81   01A3 48414600 ADD DWORD PTR DS:[EBX+464148],ESP
00420F87   83EC 10       SUB ESP,10
00420F8A   C9            LEAVE
00420F8B   C3            RETN
```

Efectivamente hay uno, ya sabemos donde conseguir la dirección de todos ellos fácilmente, probé a poner un "BP Conditional Log" y loggear EBP cada vez pero lo detectaba y cerraba el programa así que era inútil nome sirvió, ya veremos más adelante lo que haremos, de momento vamos a estudiar más la rutina en la que estamos, tenemos en EBP la dirección del próximo CALL que va a redirigir, traceemos un poco más vienen varios "CALL EDX" donde va cambiando el valor de EAX, imagino que está operando para calcular direcciones, etc… si seguimos traceando con F7 para entrar en los CALL llegamos a una zona donde parece que busca la dll a tratar, es aquí:

```
00C3A02B   833F 00       CMP DWORD PTR DS:[EDI],0
00C3A02E ^ 0F85 ABFBFFFF JNZ 00C39BDF
00C3A034   8D95 C4FEFFFF LEA EDX,DWORD PTR SS:[EBP-13C]
00C3A03A   B8 73000000   MOV EAX,73
00C3A03F   E8 08DAFFFF   CALL 00C37A4C
00C3A044   8B85 C4FEFFFF MOV EAX,DWORD PTR SS:[EBP-13C]
```

Esta es la salida del bucle, dejará de iterar cuando [EDI]=!0, es decir cuando EDI apunte a una zona donde haya ceros, si seguimos este JNZ iremos para arriba donde nos encontramos estas líneas:

```
00C39BDF   83C7 04        ADD EDI,4
00C39BE2   8B17           MOV EDX,DWORD PTR DS:[EDI]
00C39BE4   83C7 04        ADD EDI,4
00C39BE7   66:8B07        MOV AX,WORD PTR DS:[EDI]
00C39BEA   83C7 02        ADD EDI,2
00C39BED   66:3B45 F8     CMP AX,WORD PTR SS:[EBP-8]
00C39BF1 v 74 0D          JE SHORT 00C39C00
00C39BF3   83EA 02        SUB EDX,2
00C39BF6   83EA 04        SUB EDX,4
00C39BF9   03FA           ADD EDI,EDX
00C39BFB v E9 2B040000    JMP 00C3A02B
```

va aumentando el valor de EDI en 4 unidades para que apunte al siguiente DWORD, después pone el puntero de EDI en EDX, vuelve a añadirle 4 a EDI y pone el WORD bajo de su puntero en AX, le vuelve a sumar 2 unidades más y compara el valor de AX con el de [EBP-8] a todo esto cuando estamos en la línea :C39BED vemos que EDI apunta al nombre de una librería, este es un ejemplo:

```
Registers (MMX)
EAX 00C4FAF9
ECX AEE76B1A
EDX 0000002A
EBX 00000000
ESP 0022FD98
EBP 0022FEEC
ESI 00C70638
EDI 00C764C7 ASCII "COMDLG32.DLL"
EIP 00C39BED
```

Ahora apunta a la cadena ASCII "COMDLG32.DLL" y tenemos AX==FAF9 mientras que por otro lado tenemos [EBP-8]==6B1A o sea que el salto no se produce, quizá porque no encontró la DLL adecuada, seguimos en el bucle hasta que sí que la encuentre:

```
00C39BDF   83C7 04        ADD EDI,4
00C39BE2   8B17           MOV EDX,DWORD PTR DS:[EDI]
00C39BE4   83C7 04        ADD EDI,4
00C39BE7   66:8B07        MOV AX,WORD PTR DS:[EDI]
00C39BEA   83C7 02        ADD EDI,2
00C39BED   66:3B45 F8     CMP AX,WORD PTR SS:[EBP-8]
00C39BF1 v 74 0D          JE SHORT 00C39C00
00C39BF3   83EA 02        SUB EDX,2
00C39BF6   83EA 04        SUB EDX,4
00C39BF9   03FA           ADD EDI,EDX
00C39BFB v E9 2B040000    JMP 00C3A02B
00C39C00   8B46 48        MOV EAX,DWORD PTR DS:[ESI+48]
00C39C03   8B00           MOV EAX,DWORD PTR DS:[EAX]
```

Si miramos en la ventana de registros vemos:

```
Registers (MMX)
EAX 00C46B1A
ECX AEE76B1A
EDX 0000037A
EBX 00000000
ESP 0022FD98
EBP 0022FEEC
ESI 00C70638
EDI 00C765EF ASCII "KERNEL32.dll"
```

O sea que va a tratar una API del Kernel, esto es casi seguro sino no tiene sentido la comprobaición y el salto posterior que hace que se ejecuten sentencias que antes no se ejecutaban, si entramos en esta zona y vamos tranquilos con F7 sin prisas llegamos a una instrucción donde comenzamos a ver nombre de apis en el registro EAX, pasa aquí:

```
00C39F75   8BCB           MOV ECX,EBX
00C39F77   8BD7           MOV EDX,EDI
00C39F79   8B45 E4        MOV EAX,DWORD PTR SS:[EBP-1C]   <-- Pone en EAX una api..!!
00C39F7C   E8 8BC1FDFF    CALL 00C1610C
00C39F81   6A 0A          PUSH 0A
00C39F83   8D4E 12        LEA ECX,DWORD PTR DS:[ESI+12]
```

Si miramos en lla ventana de registros veremos concretamente de cuál se trata:

```
Registers (MMX)
EAX 0022FDCF ASCII "CreateSemaphoreA"
ECX 00000011
EDX 00C7660F
EBX 00000011
ESP 0022FD98
EBP 0022FEEC
ESI 00C70638
EDI 00C7660F
```

En esta ocasión se trata de la api CreateSemaphoreA, de momento sigamos a ver qué más hace, pues llega a un CALL donde le pasa los parámetros "Kerenel32" y "CreateSemaphoreA" entramos dentro y salta a la api LoadLibraryExA, el retorno queda EAX==77E40000, esto lo hace en el call de la dirección :C39F97 (en cada máquina variará aunque el offset será el mismo) después de esto se vuelve a meter en otro bucle donde parece que está buscando la dirección de la api dentro del kernel, quizá no quiera llamar a GetProcAddress o quizá esté haciendo otra cosa ¿? El inicio del bucle es este:

```
00C38CD6   8B07          MOV EAX,DWORD PTR DS:[EDI]
00C38CD8   03C3          ADD EAX,EBX
00C38CDA   8945 E4       MOV DWORD PTR SS:[EBP-1C],EAX
00C38CDD   33C9          XOR ECX,ECX
00C38CDF   55            PUSH EBP
00C38CE0   68 648DC300   PUSH 0C38D64
00C38CE5   64:FF31       PUSH DWORD PTR FS:[ECX]
```

y el final del mismo es este:

```
00C38D75   46            INC ESI
00C38D76   83C7 04       ADD EDI,4
00C38D79   8B45 E8       MOV EAX,DWORD PTR SS:[EBP-18]
00C38D7C   3B70 18       CMP ESI,DWORD PTR DS:[EAX+18]
00C38D7F   0F82 51FFFFFF JB 00C38CD6
00C38D85   8B45 F4       MOV EAX,DWORD PTR SS:[EBP-C]
00C38D88   5F            POP EDI
00C38D89   5E            POP ESI
00C38D8A   5B            POP EBX
00C38D8B   8BE5          MOV ESP,EBP
00C38D8D   5D            POP EBP
00C38D8E   C2 0400       RETN 4
```

La condición de salida está cuando ESI>=[EAX+18] y en [EAX+18] vemos el valor 3Aeh y ESI se incrementa en una unidad cada vez, pedazo de bucle no? Jeje.. no desesperaremos al final tendremos recompensa eso seguro, tenemos que asegurarnos de lo que está haciendo aunque yo juraría que está buscando la api como dije antes, sobretodo porque en al CALL de la dirección :C38CF1 pasa como parámetro el nombre de 2 apis, una que no varía "CreateSemaphoreA" en este caso y la otra que va cambiando, el retorno va siendo cada vez EAX=!0 suponemos que cuando la encuentre será EAX==0, como no detecta los HBP ponemos uno en dicho CALL, bueno mejor en el "TEST EAX,EAX" de después y hasta que no sea EAX==0 iremos pulsando F9 así sólo tracearemos el código a partir de que haya encontrado la API que busca ;-D o todavía mejor lo ponemos debajo del salto así cuando pare es que EAX==0 y nos evitamos pulsar F9 bastantes veces, cuando pare seguimos con F7 y nos saca de la rutina, buena señal esta, al final queda en EAX un puntero al nombre de la api que verifica aquí:

```
00C39FEA   8B45 FC       MOV EAX,DWORD PTR SS:[EBP-4]
00C39FED   E8 A2ECFFFF   CALL 00C38C94                    ;Aqui busca la api...
00C39FF2   8B55 08       MOV EDX,DWORD PTR SS:[EBP+8]
00C39FF5   8902          MOV DWORD PTR DS:[EDX],EAX
00C39FF7   8D85 E3FEFFFF LEA EAX,DWORD PTR SS:[EBP-11D]
00C39FFD   3B45 E4       CMP EAX,DWORD PTR SS:[EBP-1C]
00C3A000   74 08         JE SHORT 00C3A00A
00C3A002   8B45 E4       MOV EAX,DWORD PTR SS:[EBP-1C]
```

Miramos en la ventanita auxiliar y vemos:

```
Stack SS:[0022FED0]=0022FDCF, (ASCII "CreateSemaphoreA")
EAX=0022FDCF, (ASCII "CreateSemaphoreA")
```

efectivamente buscaba la api, jeje… bueno ya la tiene veamos que hace con ella:

```
00C3A7DD   E8 66F3FFFF   CALL 00C39B48
00C3A7E2   8B4424 0C     MOV EAX,DWORD PTR SS:[ESP+C]
00C3A7E6   0383 E0000000 ADD EAX,DWORD PTR DS:[EBX+E0]     ;en eax la API...
00C3A7EC   EB 01         JMP SHORT 00C3A7EF
00C3A7EE   E9 8B542414   JMP 14E7FC7E
00C3A7F3   52            PUSH EDX
00C3A7F4   6A 00         PUSH 0
00C3A7F6   8D4C24 18     LEA ECX,DWORD PTR SS:[ESP+18]
00C3A7FA   8BD0          MOV EDX,EAX
00C3A7FC   8B43 3C       MOV EAX,DWORD PTR DS:[EBX+3C]
00C3A7FF   E8 CCB6FFFF   CALL 00C35ED0
00C3A804   8BC8          MOV ECX,EAX
00C3A806   8A5424 18     MOV DL,BYTE PTR SS:[ESP+18]
00C3A80A   8BC3          MOV EAX,EBX
00C3A80C   E8 EF010000   CALL 00C3AA00
00C3A811   2BC5          SUB EAX,EBP
00C3A813   83E8 05       SUB EAX,5
00C3A816   45            INC EBP
00C3A817   8945 00       MOV DWORD PTR SS:[EBP],EAX       ;parchea los CALL
00C3A81A   8B4424 10     MOV EAX,DWORD PTR SS:[ESP+10]
00C3A81E   8B00          MOV EAX,DWORD PTR DS:[EAX]
00C3A820   894424 14     MOV DWORD PTR SS:[ESP+14],EAX
00C3A824   EB 0C         JMP SHORT 00C3A832
```

Pues nada rápidamente hemos salido del "CALL C39B48" que es donde hizo todo y pone la api en EAX pero ahora no pone ningún puntero sino que pone la verdadera api:

```
Registers (MMX)
EAX 77E504A7 kernel32.CreateSemaphoreA
ECX 0022FEEC
EDX 00000000
EBX 00C70788
ESP 0022FEF8
EBP 00420FBF 1stStart.00420FBF
ESI 00EC1025
EDI 00C707C8
```

Ahora sí que podemos casi asegurar que el call que está tratando debería ser un JMP o un CALL a la api que vemos, después de esto se pone a mirar los Opcodes:

```
00C35F52   84C0          TEST AL,AL
00C35F54 v 74 14         JE SHORT 00C35F6A
00C35F56 v EB 01         JMP SHORT 00C35F59
00C35F58   C7C6 45EF0133 MOV ESI,3301EF45
00C35F5E   C05A 59 59    RCR BYTE PTR DS:[EDX+59],59          Shift constant out of range 1.
00C35F62   64:8910       MOV DWORD PTR FS:[EAX],EDX
00C35F65 v E9 D0000000   JMP 00C3603A
00C35F6A   66:813F FF25  CMP WORD PTR DS:[EDI],25FF
00C35F6F v 75 17         JNZ SHORT 00C35F88
00C35F71 v EB 01         JMP SHORT 00C35F74
00C35F73   E8 83C7028B   CALL 8BC626FB
00C35F78   07            POP ES                               Modification of segment regist
00C35F79   8B38          MOV EDI,DWORD PTR DS:[EAX]
```

Se ve que es una zona llena de saltos, antes de esto también puso en AL el primer byte de la api y miró a ver si era 0E8h o 0E9h que son los opcode de CALL y JMP respectivamente, bueno sigamos a ver que más sorpresas hay ;-) continúa tomando el primer byte de la API y haciendo operaciones con él, comparándolo, etc… hasta que al final se mete en este importantísima llamada:

```
00C35FBB   8ACB          MOV CL,BL
00C35FBD   8BD7          MOV EDX,EDI
00C35FBF   8BC6          MOV EAX,ESI
00C35FC1   E8 4601FEFF   CALL 00C1610C                        ;ojo a esta llamada...
00C35FC6   33C0          XOR EAX,EAX
00C35FC8   8AC3          MOV AL,BL
```

Entramos dentro, vemos un subcall en el que entramos también y que es este:

```
00C126A4   56            PUSH ESI
00C126A5   57            PUSH EDI
00C126A6   89C6          MOV ESI,EAX
00C126A8   89D7          MOV EDI,EDX
00C126AA   89C8          MOV EAX,ECX
00C126AC   39F7          CMP EDI,ESI
00C126AE v 77 13         JA SHORT 00C126C3
00C126B0 v 74 2F         JE SHORT 00C126E1
00C126B2   C1F9 02       SAR ECX,2
00C126B5 v 78 2A         JS SHORT 00C126E1
00C126B7   F3:A5         REP MOVS DWORD PTR ES:[EDI],DWORD PTR DS:[ESI]
00C126B9   89C1          MOV ECX,EAX
00C126BB   83E1 03       AND ECX,3
00C126BE   F3:A4         REP MOVS BYTE PTR ES:[EDI],BYTE PTR DS:[ESI]
00C126C0   5F            POP EDI
00C126C1   5E            POP ESI
00C126C2   C3            RETN
```

cuando llegamos al "REP MOVS…" donde va a copiar código de ESI a EDI miramos en la ventana de registros a ver qué valores tienen ambos y vemos:

```
Registers (3DNow!)
EAX 00000001
ECX 00000000
EDX 00C77768
EBX 00000001
ESP 0022FEA4
EBP 0022FEE8
ESI 77E504A7 kernel32.CreateSemaphoreA
EDI 00C77768
```

Con que ESI tiene un puntero a la api que antes encontró "CreateSemaphoreA" y en EDI tiene un puntero a una zona libre como se puede ver en la imagen:

```
Address   Hex dump                                          ASCII
00C77768  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
00C77778  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
00C77788  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
00C77798  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
```

No hay absolutamente nada allí, ejecutamos el "REP MOVS…" y empieza a aparecer código allí, lo ponemos en modo "Disassemble" y vemos las instrucciones tal cual son, primero aparece un "PUSH EBP" salimos del call y fuera encontramos una comparación entre EAX y 6Ceh como es EAX==1 es inferior salta al inicio de la rutina:

```
00C35FBD   8BD7          MOV EDX,EDI
00C35FBF   8BC6          MOV EAX,ESI
00C35FC1   E8 4601FEFF   CALL 00C1610C                        ;ojo a esta llamada...
00C35FC6   33C0          XOR EAX,EAX
00C35FC8   8AC3          MOV AL,BL
00C35FCA   03F0          ADD ESI,EAX
00C35FCC   33C0          XOR EAX,EAX
00C35FCE   8AC3          MOV AL,BL
00C35FD0   03F8          ADD EDI,EAX
00C35FD2   8BC6          MOV EAX,ESI
00C35FD4   2B45 E4       SUB EAX,DWORD PTR SS:[EBP-1C]
00C35FD7   3D CE060000   CMP EAX,6CE
00C35FDC v 76 0A         JBE SHORT 00C35FE8
00C35FDE   33C0          XOR EAX,EAX
00C35FE0   5A            POP EDX
00C35FE1   59            POP ECX
00C35FE2   59            POP ECX
00C35FE3   64:8910       MOV DWORD PTR FS:[EAX],EDX
00C35FE6 v EB 52         JMP SHORT 00C3603A
00C35FE8   33C0          XOR EAX,EAX
00C35FEA   5A            POP EDX
00C35FEB   59            POP ECX
00C35FEC   59            POP ECX
00C35FED   64:8910       MOV DWORD PTR FS:[EAX],EDX
00C35FF0 ^ E9 09FFFFFF   JMP 00C35EFE                         ;vamos al inicio del loop...
```

subimos para arriba y se vuelve a ejcutar el código que acabamos de estudiar, si nos vamos fijando en el REP MOVS va copiando la api a otra zona que crea en tiempo de ejecución y que pidió no hará mucho mediante la api VirtualAlloc:

```
Address   Hex dump   Disassembly                          Comment
00C77754  55         PUSH EBP
00C77755  8BEC       MOV EBP,ESP
00C77757  51         PUSH ECX
00C77758  51         PUSH ECX
00C77759  56         PUSH ESI
00C7775A  33F6       XOR ESI,ESI
00C7775C  3975 14    CMP DWORD PTR SS:[EBP+14],ESI
00C7775F  0000       ADD BYTE PTR DS:[EAX],AL
00C77761  0000       ADD BYTE PTR DS:[EAX],AL
00C77763  0000       ADD BYTE PTR DS:[EAX],AL
00C77765  0000       ADD BYTE PTR DS:[EAX],AL
```

Vemos las primeras instrucciones de la misma, es parte del código de la api "CreateSempahoreA" como podemos comprobar fácilmente, hacemos esto:

```
Command : ?CreateSemaphoreA        HEX: 77E504A7 - DEC: 2011497639 - ASCII: w¥ • §
```

Obtenemos la dirección real de la api en nuestra máquina, vamos para allá con Ctr+G:

```
77E504A7  55          PUSH EBP
77E504A8  8BEC        MOV EBP,ESP
77E504AA  51          PUSH ECX
77E504AB  51          PUSH ECX
77E504AC  56          PUSH ESI
77E504AD  33F6        XOR ESI,ESI
77E504AF  3975 14     CMP DWORD PTR SS:[EBP+14],ESI
77E504B2  0F85 91000000  JNZ kernel32.77E50549
77E504B8  56          PUSH ESI
77E504B9  FF75 10     PUSH DWORD PTR SS:[EBP+10]
77E504BC  FF75 0C     PUSH DWORD PTR SS:[EBP+C]
77E504BF  FF75 08     PUSH DWORD PTR SS:[EBP+8]
77E504C2  E8 44FAFFFF CALL kernel32.CreateSemaphoreW
77E504C7  5E          POP ESI
```

Como vemos coinciden las instrucciones, lo que acaba de hacer es copiar parte de la API, jeje… lo que está tramando este canalla ;-D Cuando termina de copiar parte de la api (lo que ve él oportuno) inserta un PUSH a mitad de api y un RETN para que nos lleve allí y se siga ejecutando con normalidad:

```
00C3605A  8D56 01       LEA EDX,DWORD PTR DS:[ESI+1]
00C3605D  8902          MOV DWORD PTR DS:[EDX],EAX
00C3605F  83C6 05       ADD ESI,5
00C36062  C606 68       MOV BYTE PTR DS:[ESI],68          ;inserta un PUSH...
00C36065  807D EF 00    CMP BYTE PTR SS:[EBP-11],0
00C36069  74 0A         JE SHORT 00C36075
00C3606B  8D46 01       LEA EAX,DWORD PTR DS:[ESI+1]
00C3606E  8B55 E8       MOV EDX,DWORD PTR SS:[EBP-18]
00C36071  8910          MOV DWORD PTR DS:[EAX],EDX
00C36073  EB 05         JMP SHORT 00C3607A
00C36075  8D46 01       LEA EAX,DWORD PTR DS:[ESI+1]
00C36078  8938          MOV DWORD PTR DS:[EAX],EDI        ;reajusta el PUSH anterior...
00C3607A  8D46 05       LEA EAX,DWORD PTR DS:[ESI+5]
00C3607D  C600 C3       MOV BYTE PTR DS:[EAX],0C3         ;inserta un RETN...
00C36080  83C6 06       ADD ESI,6
00C36083  EB 01         JMP SHORT 00C36086
```

Ahí está como primero inserta un "PUSH 0" después lo reajusta para que apunte donde debe y finalmente inserta también el RETN final, queda así:

```
Address   Hex dump   Disassembly                          Comment
00FF0000  55         PUSH EBP
00FF0001  8BEC       MOV EBP,ESP
00FF0003  51         PUSH ECX
00FF0004  51         PUSH ECX
00FF0005  56         PUSH ESI
00FF0006  33F6       XOR ESI,ESI
00FF0008  3975 14    CMP DWORD PTR SS:[EBP+14],ESI
00FF000B  68 B204E577  PUSH 77E504B2
00FF0010  C3         RETN
```

pues ya está, hizo el tabajo de copiar la api y saltar a mitad de la misma pero todavía falta algo no? Jeje.. claro reajustal el CALL para que no apunte a la api directamente sino que apunte a esta zona, esto lo hace justo después, pero antes prepara una segunda zona que pide con VirtualAllocEx, allí copia una o dos instrucciones y luego inserta un PUSH a la zona :0FF0000 donde copió parte de la API, esto lo hace aquí:

```
00C3A9B1  8B45 08       MOV EAX,DWORD PTR SS:[EBP+8]
00C3A9B4  8B40 FC       MOV EAX,DWORD PTR DS:[EAX-4]
00C3A9B7  C600 68       MOV BYTE PTR DS:[EAX],68          ;inserta el PUSH...
00C3A9BA  8B45 08       MOV EAX,DWORD PTR SS:[EBP+8]
00C3A9BD  8B40 F8       MOV EAX,DWORD PTR DS:[EAX-8]
00C3A9C0  8B55 08       MOV EDX,DWORD PTR SS:[EBP+8]
00C3A9C3  8B52 FC       MOV EDX,DWORD PTR DS:[EDX-4]
00C3A9C6  42            INC EDX
00C3A9C7  8902          MOV DWORD PTR DS:[EDX],EAX        ;reajusta el PUSH...
00C3A9C9  8B45 08       MOV EAX,DWORD PTR SS:[EBP+8]
00C3A9CC  8B40 FC       MOV EAX,DWORD PTR DS:[EAX-4]
00C3A9CF  83C0 05       ADD EAX,5
00C3A9D2  C600 C3       MOV BYTE PTR DS:[EAX],0C3         ;inserta el RETN...
00C3A9D5  5D            POP EBP
00C3A9D6  C3            RETN
```

Y deja una segunda zona preparada donde ejecuta algo de código y luego salta a :0FF0000 mediante un PUSH-RETN, veámoslo:

```
Address   Hex dump    Disassembly                      Comment
01000004  FF0424      INC DWORD PTR SS:[ESP]
01000007  68 0000FF00 PUSH 0FF0000
0100000C  C3          RETN
0100000D  0000        ADD BYTE PTR DS:[EAX],AL
0100000F  0000        ADD BYTE PTR DS:[EAX],AL
```

Así queda finalmente, ejecuta el "INC DWORD PTR [ESP]" y después salta a :0FF0000 donde está parte de la api a ejcutar copiada, esto lo hará imagino para complicar más las cosas, para una simple api crea dos secciones en tiempo de ejecución para realmente poco código pero bueno, esta es la manera en que trabaja las apis, cuando ha hecho esto ahora sí parchea el CALL para que apunte a la segunda zona creada :01000004, es aquí:

```
00C3A80A   8BC3          MOV EAX,EBX
00C3A80C   E8 EF010000   CALL 00C3AA00
00C3A811   2BC5          SUB EAX,EBP                  1stStart.00420F7C
00C3A813   83E8 05       SUB EAX,5
00C3A816   45            INC EBP
00C3A817   8945 00       MOV DWORD PTR SS:[EBP],EAX   ;aqui parchea el CALL...
00C3A81A   8B4424 10     MOV EAX,DWORD PTR SS:[ESP+10]
00C3A81E   8B00          MOV EAX,DWORD PTR DS:[EAX]
00C3A820   894424 14     MOV DWORD PTR SS:[ESP+14],EAX
00C3A824 v EB 0C         JMP SHORT 00C3A832
00C3A826   8B43 2C       MOV EAX,DWORD PTR DS:[EBX+2C]
```

Lo que hace es bien sencillo, para entenderlo mejor veamos los registros:

```
Registers (FPU)
EAX 01000004
ECX 0022FEF0
EDX 01000008
EBX 00C70788
ESP 0022FEF8
EBP 00420F7C 1stStart.00420F7C
ESI 00EC0FED
EDI 00C707C8
```

tenemos en EAX la dirección de la segunda zona a la que deberá apuntar el CALL, en EBP la dirección donde está el CALL a parchear, entonces al restar EBP a EAX queda en EAX los bytes que deberán ir en la dirección del CALL+1 para que aupnte a :01000004, la dirección del CALL+1 porque el primer byte es el Opcode 0E8h y no se debe modificar por esto antes del "MOV DWORD PTR [EBP], EAX" hace el "INC EBP" y después sigue con el mismo trabajo para los demás CALL's.

Pues bien hemos visto más o menos como trabaja este ASProtect el tema de las API's espero haberlo explicado bien, con todo esto estamos ya preparados para fundir la protección, jejeje… este es el paso más importante ;-)

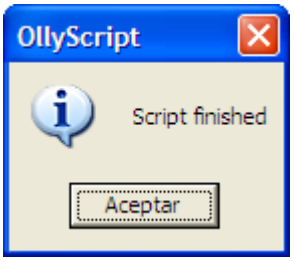
He de decir que soy muy mal programador y ni si quiera se hacer un simple script a no ser que sea muy sencillo, uno que sea má experto podría hacerse un script que vaya tomando la dirección de la API y la del CALL para reemplazarlos por saltos a las apis directamente por medio de la Iat claro está, o sea el salto "JMP DWORD PTR [xxxxxxxx]" donde las xxxx son una dirección de la IAT, justamente la que ocupa la API a la que debe de ir, pero lamentablemente como digo no soy un buen programador y esto escapa de mis posibilidades ;-(No obstante esto no nos va a detener para nada ya que hay muchas maneras de llegar al mismo sitio.

Lo que vamos a hacer de momento es obtener la dirección del CALL y la dirección de la API correcta a la que deberá de apuntar, esto se puede hacer con un BPConditional Log pero lo detecta y detiene la ejecución, entonces otra opción es este script:

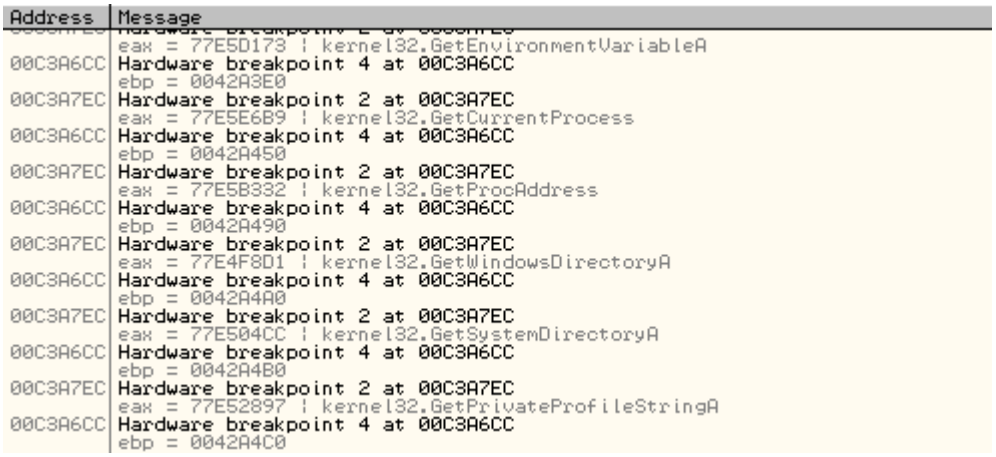
```
-------------/ inicio_script
start:
eob break
run

break:
cmp eip, 00C3A6CC
jne uno
log ebp
jmp start

uno:
cmp eip, 00C3A7EC
jne fin
log eax
```

```
jmp start

fin:
ret
-------------/ final_script
```

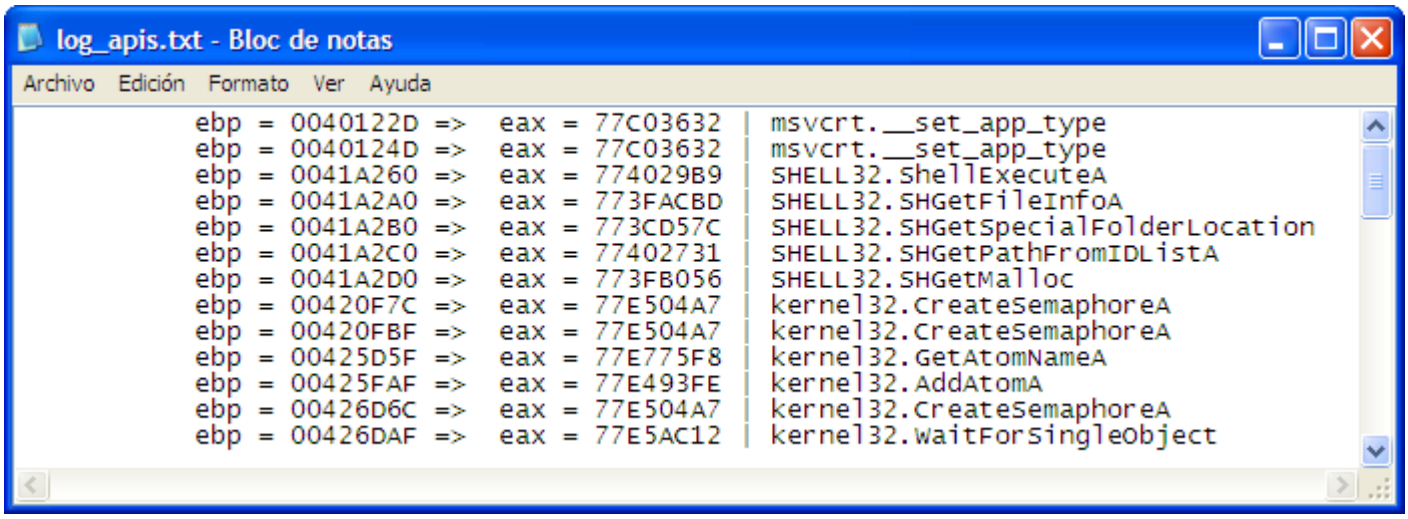
Lo que hace es muy sencillito, primero de todo ponemos un HBP en :C3A6CC y otro en :C3A7EC para que el script funcione, una vez allí lo único que hace es logguear el EBP en el primer punto (dirección del CALL) y loguear EAX en el segundo punto (dirección de la api a la que debe apuntar el CALL), reiniciamos Olly y destildamos las excepciones para que no se detenga en ninguna, pulsamod F9 para dejar correr el programa y cuando termine el script:



el programa ya está ejecutándose miramos la ventanita Log:



Allí tenemos los datos, jeje.. la dirección del CALL y la api a la que debería de ir, guardamos la tabla entera, picamos con el botón derecho y elegimos la opción "copy to clipboard->Whole table" y la dejamos así:



Y ya temnemos parte del trabajo ;-D

Como ya he dicho aquí vamos a trabajar bastante porque las vamos a reparar todas manualmente, si alguien tiene un método se lo agradecería mucho, explico la primera y las siguiente son igual:

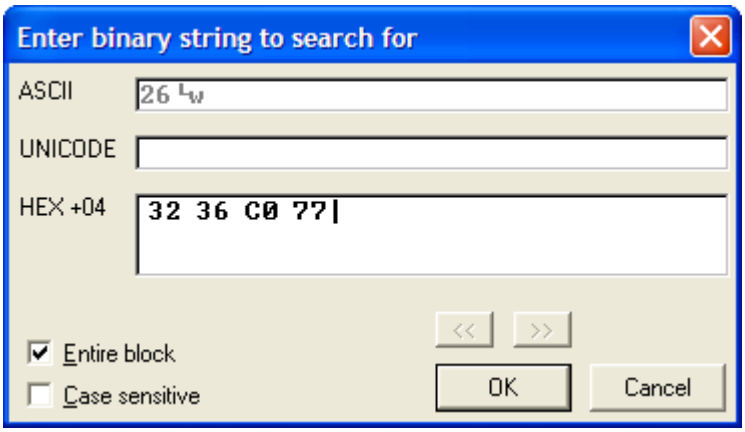
Vamos a la dirección del primer CALL que es :40122D y allí lo vemos:

```
0040122D   E8 D2EDB100   CALL 00F20004
00401232   CF            IRETD
00401233   E8 C8FEFFFF   CALL 1stStart.00401100
00401238   90            NOP
```

Vemos que apunta a :00F20004 una dirección que no existirá en el dumpeado ya que la crea ASProtect en tiempo de ejecución, entoces debemos reparar estos CALL, gracias al Log sabemos que debería apuntar a "msvcrt.__set_app_type" entonces buscamos la dirección en la IAT de esta API, pero antes la dirección:

```
Command : ?msvcrt.__set_app_type    HEX: 77C03632 - DEC: 2009085490 - ASCII: wÀ62
```

Ahora la buscamos en la IAT con Ctrl+B y ponemos la dirección pero al revés:



Pulsamos OK y nos dice donde está ;-DD sobretodo hay que tener en cuenta que hay que hacer la búsqueda en la sección que alberga la IAT:

Address	Size	Owner	Section	Contains	Type	Access	Initial	Mapped as
003E0000	00004000				Priv	RW	RW	
003F0000	00003000				Map	R	R	\Device\HarddiskVolume1\WINDOW
00400000	00001000	1stStart		PE header	Imag	R	RWE	
00401000	00058000	1stStart		code	Imag	R	RWE	
00459000	00001000	1stStart		data	Imag	R	RWE	
0045A000	00006000	1stStart			Imag	R	RWE	
00460000	00005000	1stStart			Imag	R	RWE	
00465000	00002000	1stStart			Imag	R	RWE	
00467000	00028000	1stStart	.rsrc	resources	Imag	R	RWE	
0048F000	00023000	1stStart	.data	imports,rel	Imag	R	RWE	
004B2000	00001000	1stStart	.adata		Imag	R	RWE	
004C0000	00041000				Map	R	R	\Device\HarddiskVolume1\WINDOW

hacemos un "Dump in CPU" para tener los bytes de la sección a la vista en la ventana de datos entonces ya podemos hacer la búsqueda, encontramos los bytes aquí:

Address	Hex dump	ASCII
00465634	32 36 C0 77 DA 37 C0 77 00 7B C0 77 90 28 BE 77	26 w 7 w { w (w
00465644	D4 EB BE 77 D7 C3 BF 77 78 DC BE 77 FC DC BE 77	w w x w w w
00465654	D7 DD BE 77 87 CA BF 77 37 EE BE 77 80 AC C2 77	w w 7 w w
00465664	CD C2 BE 77 0A DA BF 77 F6 30 C0 77 88 A9 C2 77	w w w 0 w w
00465674	7E E4 BF 77 8C 5B C1 77 F5 24 C1 77 FC 5B C1 77	w w [w $ w [w

O sea que los bytes de la api "msvcrt.__set_app_type" se encuentran en la dirección :465634, entonces debemos cambiar el "CALL 00F20004" por:

CALL DWORD PTR [465634]

De manera que se haga una llamada a la IAT directamente en lugar de la llamada a la zona de ASProtect, quedará de esta manera:

```
00401220   55            PUSH EBP
00401221   89E5          MOV EBP,ESP
00401223   83EC 08       SUB ESP,8
00401226   C70424 0100000( MOV DWORD PTR SS:[ESP],1
0040122D   FF15 34564600 CALL DWORD PTR DS:[<&msvcrt.__set_app_type>] msvcrt.__set_app_type
00401233   E8 C8FEFFFF   CALL dump_1.00401100
00401238   90            NOP
```

Y ya tenemos una entrada resuelta, hacemos esto con las restantes, no son muchas podría ser peor, vamos reparando los CALL que nos ha puesto ASProtect reemplazándolos por llamadas directas a la IAT, conociendo la API (gracias al Log) es fácil encontrar su dirección en la IAT y redirigir la llamada hacia allí, pero antes de continuar necesitamos saber algo más, hay calls de los puestos por el packer que se deben reemplazar por calls a la IAT y otros por JMP's a la IAT ¿Cómo distinguirlos? Muy

fácil: En realidad cuando existe la famosa JUMP_TABLE o tabla de saltos donde están todos los JMP's a la IAT juntos no hay problema ¿pero porqué son JMP y no CALL? La respuesta es porque cuando hacemos una llamada a cualquier API cuando esta finalice su trabajo llegará a un RET que nos devolverá al código del programa justo después de donde se llamó a dicha API, pero si hacemos esto por ejemplo:

```
Código…
Código…
JMP DWORD PTR [posición_iat]
Código…
Código…
```

Así sin más nos creará un problema ya que ese JMP a la IAT no mete la dirección de retorno en la pila y cuando la API termine su trabajo no sabrá donde retornar, para esto se deberá de hacer así:

```
Código…
Código…
CALL DWORD PTR [posición_iat]
Código…
Código…
```

Ya que este CALL si que empuja la dirección de retorno a la pila, entonces ¿cómo que existen JMP's a la IAT? Pues porque a ese JMP se llega desde un CALL que es el que empuja la dirección previamente, es decir como ejemplo:

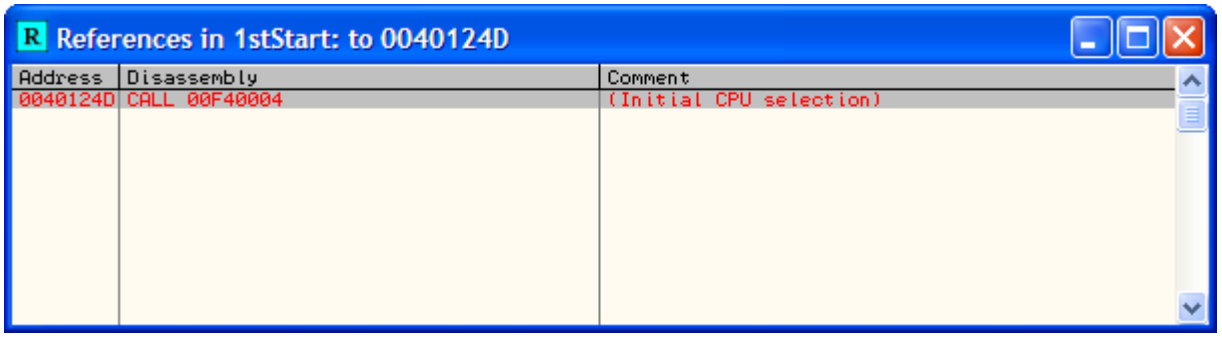
```
403550 Código…
403554 Código…
40355A CALL 401210      /*empuja a la pila :403560*/
403560 Código…
403566 Código…


4011F3 Código…
4011FF Código…
401210 JMP DWORD PTR [posición_iat]
401216 Código…
401219 Código…
```

Salta a la API y cuando termina retorna a :403560 que es la dirección de retorno, fijáos como utilizamos un JMP sin descuadrar la pila, esto es muy importante y un dato a tener en cuenta para reparar los CALL de ASProtect, sabido esto podemos continuar, pondremos un ejemplo de cada caso para que quede todavía más claro:

Esto es código del programa que estamos desempakando, o sea que nos vale realmente no es ningún ejemplo, empezamos con este CALL que está jodido claro:

```
00401240  55              PUSH EBP
00401241  89E5            MOV EBP,ESP
00401243  83EC 08         SUB ESP,8
00401246  C70424 0200000  MOV DWORD PTR SS:[ESP],2
0040124D  E8 B2EDB300     CALL 00F40004
00401252  4F              DEC EDI
00401253  E8 A8FEFFFF     CALL 1stStart.00401100
00401258  90              NOP
```

Es el primero que nos encontramos después del OEP, esto en principio no sabemos si debería ser un JMP o un CALL a la Iat, salgamos de dudas, nos ponemos sobre el call y buscamos referencias con Ctrl+R a ver que sale:



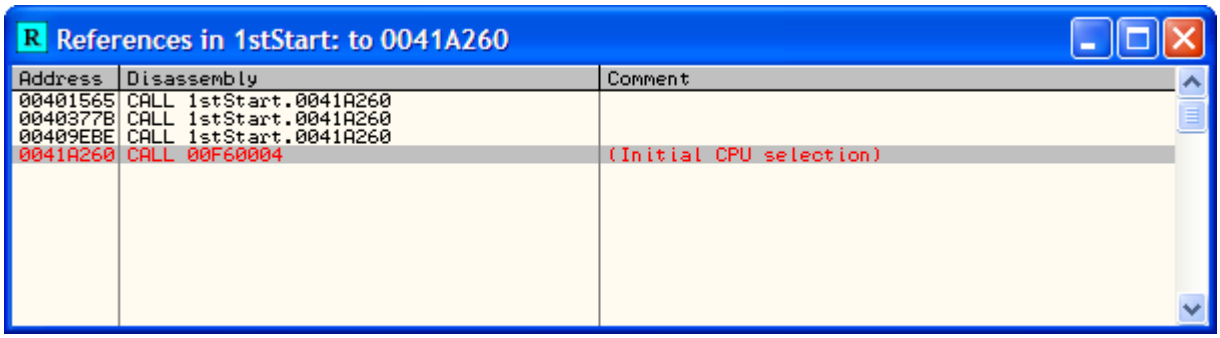
No sale nada lo cual quiere decir que no hay ningún CALL apuntando a esta dirección, o sea que deberá ser reparado como CALL a la IAT, buscamos con el Log la API a la que debe apuntar y buscamos su posición en la IAT como antes hicimos está explicado, cuando lo tenemos todo a punto reparamos dicho CALL de esta manera:

```
00401238   90              NOP
00401239   8DB426 0000000  LEA ESI,DWORD PTR DS:[ESI]
00401240   55              PUSH EBP
00401241   89E5            MOV EBP,ESP
00401243   83EC 08         SUB ESP,8
00401246   C70424 0200000  MOV DWORD PTR SS:[ESP],2
0040124D   FF15 34564600   CALL DWORD PTR DS:[465634]      msvcrt.__set_app_type
00401253   E8 A8FEFFFF     CALL 1stStart.00401100
00401258   90              NOP
```

Queda reparado como un CALL que es como debe de ser, si ponemos un JMP ya sabemos las consecuencias, jeje… así que no lo haremos, sigamos con un ejemplo contrario, o sea cuando hay que poner un JMP, fijémonos en este CALL:

```
0041A24E   0000            ADD BYTE PTR DS:[EAX],AL
0041A250 - FF25 40574600   JMP DWORD PTR DS:[465740]       olepro32.OleLoadPicture
0041A256   90              NOP
0041A257   90              NOP
0041A258   0000            ADD BYTE PTR DS:[EAX],AL
0041A25A   0000            ADD BYTE PTR DS:[EAX],AL
0041A25C   0000            ADD BYTE PTR DS:[EAX],AL
0041A25E   0000            ADD BYTE PTR DS:[EAX],AL
0041A260   E8 9F5DB400     CALL 00F60004
0041A265   1890 90000000   SBB BYTE PTR DS:[EAX+90],DL
0041A26B   0000            ADD BYTE PTR DS:[EAX],AL
0041A26D   0000            ADD BYTE PTR DS:[EAX],AL
0041A26F   00FF            ADD BH,BH
0041A271   25 4C574600     AND EAX,46574C
0041A276   90              NOP
0041A277   90              NOP
0041A278   0000            ADD BYTE PTR DS:[EAX],AL
0041A27A   0000            ADD BYTE PTR DS:[EAX],AL
0041A27C   0000            ADD BYTE PTR DS:[EAX],AL
0041A27E   0000            ADD BYTE PTR DS:[EAX],AL
0041A280 - FF25 54574600   JMP DWORD PTR DS:[465754]       SHELL32.DragQueryFileA
0041A286   90              NOP
0041A287   90              NOP
0041A288   0000            ADD BYTE PTR DS:[EAX],AL
0041A28A   0000            ADD BYTE PTR DS:[EAX],AL
0041A28C   0000            ADD BYTE PTR DS:[EAX],AL
0041A28E   0000            ADD BYTE PTR DS:[EAX],AL
0041A290 - FF25 50574600   JMP DWORD PTR DS:[465750]       SHELL32.DragFinish
0041A296   90              NOP
0041A297   90              NOP
0041A298   0000            ADD BYTE PTR DS:[EAX],AL
0041A29A   0000            ADD BYTE PTR DS:[EAX],AL
0041A29C   0000            ADD BYTE PTR DS:[EAX],AL
0041A29E   0000            ADD BYTE PTR DS:[EAX],AL
0041A2A0   E8 5F5DB600     CALL 00F80004
0041A2A5   D990 90000000   FST DWORD PTR DS:[EAX+90]
0041A2AB   0000            ADD BYTE PTR DS:[EAX],AL
0041A2AD   0000            ADD BYTE PTR DS:[EAX],AL
```

El que está seleccionado es el próximo que repararemos, aunque ya de momento se distingue la tabla de saltos de la que hablábamos antes, se ven varios JMP DWORD PTR… así que seguramente será un JMP lo que deberemos poner, aún así seguiremos con el método explicado para asegurarnos, nos situamos en la dirección del CALL a reparar :41A260 y picamos Ctrl+R nos sale esto:



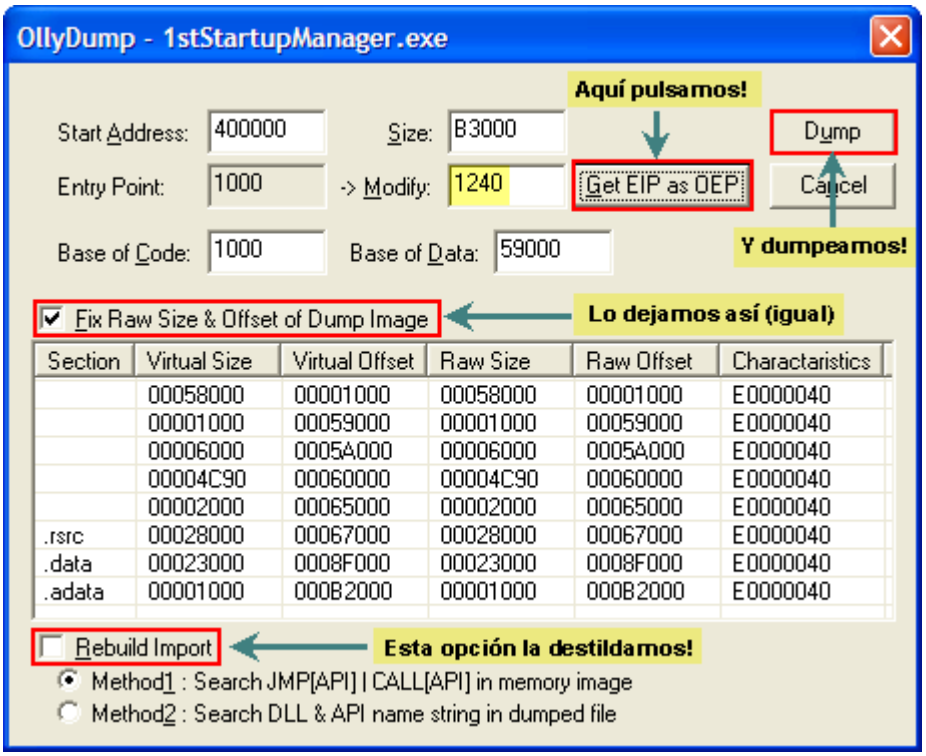
Aquí sí que hay calls apuntando, entonces debemos poner un JMP en lugar de un CALL como decíamos antes, quedará de esta manera:

```
0041A25E   0000            ADD BYTE PTR DS:[EAX],AL
0041A260 - FF25 68574600   JMP DWORD PTR DS:[465768]       SHELL32.ShellExecuteA
0041A266   90              NOP
0041A267   90              NOP
0041A268   90              NOP
0041A269   90              NOP   <---- Cuidado con estos NOP's
0041A26A   90              NOP
0041A26B   0000            ADD BYTE PTR DS:[EAX],AL
0041A26D   0000            ADD BYTE PTR DS:[EAX],AL
0041A26F   00FF            ADD BH,BH
0041A271   25 4C574600     AND EAX,46574C
0041A276   90              NOP
0041A277   90              NOP
```

Reparamos el CALL y hay que tener cuidado con esos NOP's que insertó de más, he señalado en la imagen en verde los bytes del siguiente "JMP DWORD PTR…" que no aparece por culpa precisamente de los NOP's que descuadraron todo, no creo que aquí hubiera problema ya que hay un CALL que apunta a la dirección exacta del JMP y funcionaría bien pero vamos a hacer bien las cosas, seleccionamos los NOP's y hacemos un Undo Selection:

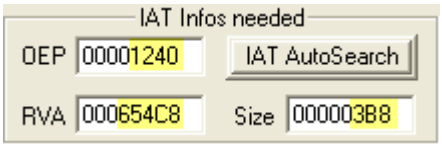
```
0041A25A   0000          ADD BYTE PTR DS:[EAX],AL
0041A25C   0000          ADD BYTE PTR DS:[EAX],AL
0041A25E   0000          ADD BYTE PTR DS:[EAX],AL
0041A260 - FF25 68574600 JMP DWORD PTR DS:[465768]      SHELL32.ShellExecuteA
0041A266   90            NOP
0041A267   90            NOP
0041A268   0000          ADD BYTE PTR DS:[EAX],AL
0041A26A   0000          ADD BYTE PTR DS:[EAX],AL
0041A26C   0000          ADD BYTE PTR DS:[EAX],AL
0041A26E   0000          ADD BYTE PTR DS:[EAX],AL
0041A270 - FF25 4C574600 JMP DWORD PTR DS:[46574C]      SHELL32.DragAcceptFiles
0041A276   90            NOP
0041A277   90            NOP
0041A278   0000          ADD BYTE PTR DS:[EAX],AL
0041A27A   0000          ADD BYTE PTR DS:[EAX],AL
```

Ahora sí que está bien hecho, vemos como va tomando forma de JMP_TABLE ;-DD sólo queda hacer lo mismo en los restantes CALLS sin más problemas, cuando los tengamos todos dumpeamos con el mismo Ollydump y estas opciones:



Estas son las opciones con las que debemos dumpear, antes de pulsar en "Dump" deberemos pulsar una vez en "Get EIPas OEP" para que nos deje el OEP correcto, cuando tenemos el dumpeado sólo nos queda reparar la IAT ;-DD

Este trabajo se lo dejamos al Import Reconstuctor que va a las mil maravillas sinceramente no utilizo el Revirgin teniendo el ImpRec, primero de todo le ponemos los datos correctos del OEP y la IAT, son estos:



En el campo OEP ponemos el offset del OEP que obtenemos de esta manera:

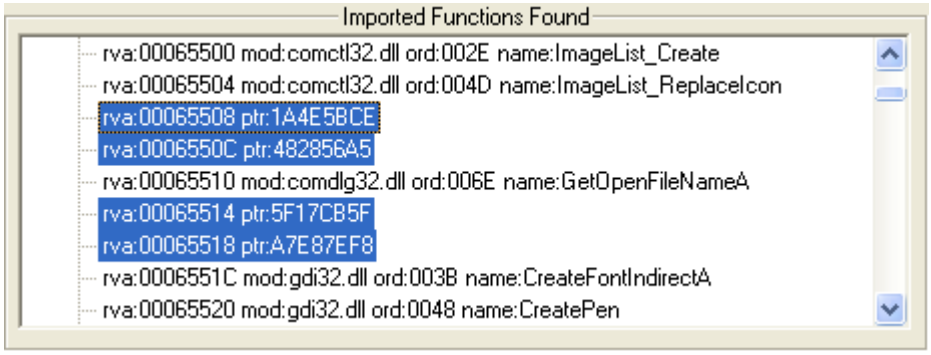
4001240 - 400000 = 1240

le restamos el imagebase del archivo (400000) que podemos mirar desde el mismo Olly, con PE Editor, etc., hay mil formas de obtenerlo y nos queda el offset '1240' este es el valor que ponemos donde pone OEP, ocurre lo mismo con el campo RVA donde va el offset del inicio de la IAT, como empieza en :4654C8 restándolo 400000 nos queda 654C8

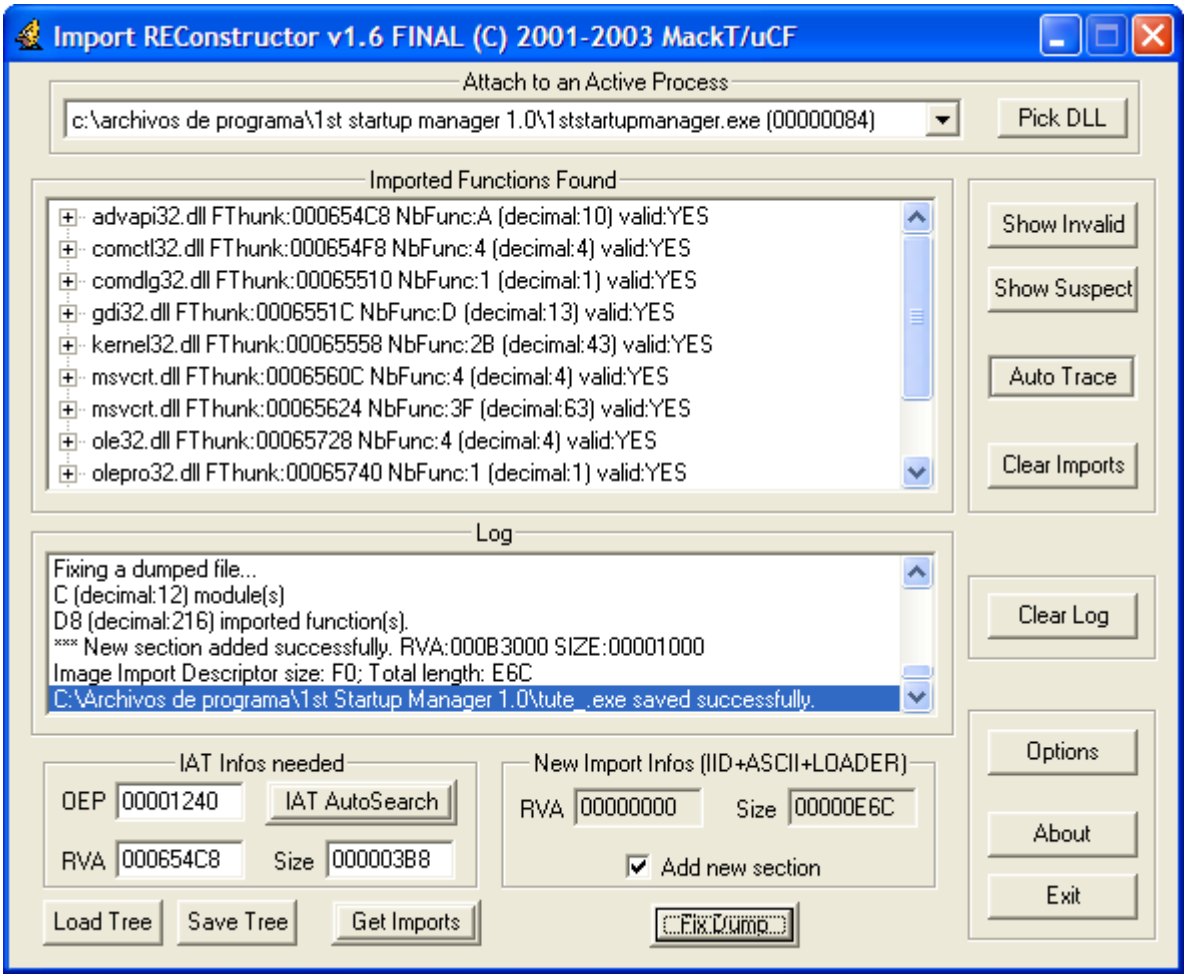
que es el dato que ponemos, y finalmente el Size que es el tamaño, restamos el inicio del final y lo obtenemos:

465880 - 4654C8 = 3B8

rellnamos los campos y pulsamos "Get Imports" nos dice que quedaron 22 entradas por resolver ¿? Muy raro esto no debería haber tantas, quizá un par de ellas (que serían GetProcAddress seguramente) pero 22¿? son muchas, demasiadas, veámoslas "Show invalid"

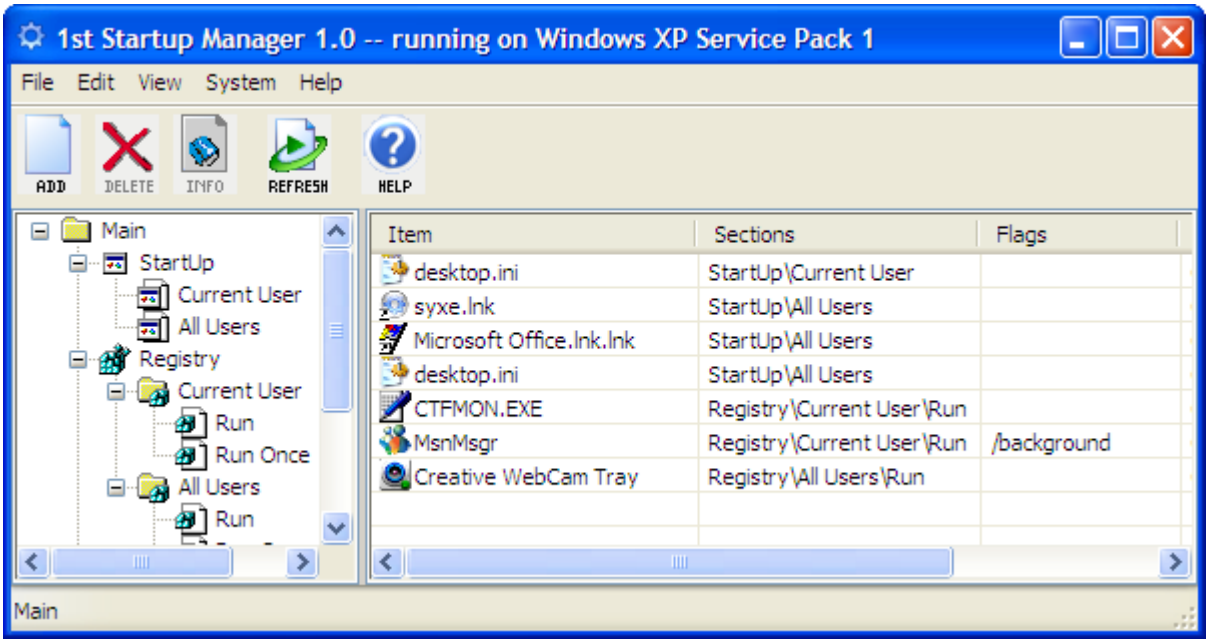


Vale vale son la separación entre dlls distintas "Cut Thunks" y se reparan todas de un plumazo, jeje… primero hemos revisado que no haya ninguna buena cuidado con esto, al terminar hacemos "Fix dump" y elegimos nuestro dumpeado que llamamos tute.exe:

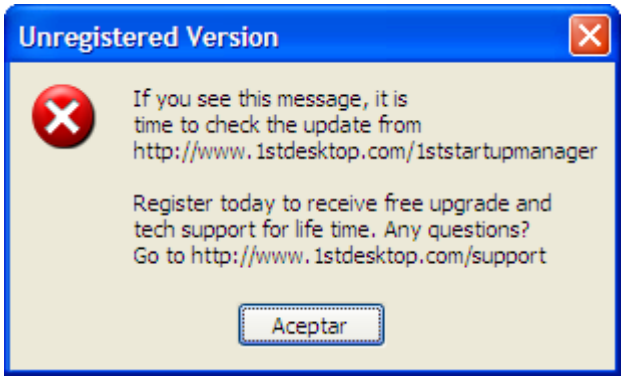


Termina su trabajo, vemos como al lado de cada dll dice valid:YES esto significa que las entradas de ese bloque están correctas, bien nos creó el ejecutable tute_.exe que es el que tiene la IAT correcta ;-DDD

Sólo quieda lo más importante de todo probarlo, a ver que ocurre con estos packers nunca se sabe cuando acaba uno el trabajo le sale un antidump, veamos ejecutaremos lógicamente tute_.exe le hacemos doble clic y…



Bieeeeeeeeeeeen por nosotros ;-DDDDDD el programa corre bien jeje.. al final valió la pena tanto trabajo, tenemos el ejecutable desempakado y podemos meterle mano al código, vamos a adelantar el reloj a ver si expira, ejecutamos el original y…



El programa expira, ejecutamos el dumpeado y corre, jeje… da la limitación temporal se encargaba el propio ASProtect y como lo hemos quitado queda libre a esto se le llama "matar dos pájaros de un tiro" muy bien ;-DD

He de decir que para poder desempakar el ejecutable nos deben quedar días del período Trial, de nada vale andar adelantando y/o atrasando el reloj ya que no funciona, seguramente tendrá en el registro sus indicadores pero no importa vamos a ver como evitar que expire para poder desempkarlo igual aunque no nos queden días del Trial, primero de nada vamos superando excepciones hasta que pasada esta aparece el mensaje:

```
00C3FE19   0156 00          ADD DWORD PTR DS:[ESI],EDX
00C3FE1C   A3 057536EB      MOV DWORD PTR DS:[EB367505],EAX
00C3FE21   019A 67648F06    ADD DWORD PTR DS:[EDX+68F6467],EBX
00C3FE27   0000             ADD BYTE PTR DS:[EAX],AL
00C3FE29   83C4 04          ADD ESP,4
00C3FE2C   C1D6 93          RCL ESI,93                          Shift constant out of rang
00C3FE2F   037424 38        ADD ESI,DWORD PTR SS:[ESP+38]
```

Cuando la pasamos nos tira a la calle, vale reiniciamos Olly y nos detenemos en esta excepción, antes de superarla nos vamos a la ventana de memoria y ponemos un BPm en la sección de ASProtect en la que está trabajando (C10000 – C54000 en mi máquina) y luego sí la superamos normal, Olly se detiene aquí:

```
00C3FDDD   53               PUSH EBX
00C3FDDE   EB 02            JMP SHORT 00C3FDE2
00C3FDE0   CD 20            INT 20
00C3FDE2   0BDA             OR EBX,EDX
00C3FDE4   8B5C24 10        MOV EBX,DWORD PTR SS:[ESP+10]
00C3FDE8   8D5B 66          LEA EBX,DWORD PTR DS:[EBX+66]
00C3FDEB   8343 52 06       ADD DWORD PTR DS:[EBX+52],6
00C3FDEF   EB 02            JMP SHORT 00C3FDF3
00C3FDF1   CD 20            INT 20
```

O sea que es la primera instrucción que se ejecuta tras la excepción, tras superar el RET nos mete en ntdll.dll com otenemos el BPM damos RUN y para aquí:

```
00C3FE1F  36:EB 01        JMP SHORT 00C3FE23                  Superfluous prefix
00C3FE22  9A 67648F06 00  CALL FAR 0000:068F6467              Far call
00C3FE29  83C4 04         ADD ESP,4
00C3FE2C  C1D6 93         RCL ESI,93                          Shift constant out of rang
00C3FE2F  037424 38       ADD ESI,DWORD PTR SS:[ESP+38]
00C3FE33  5E              POP ESI
00C3FE34  A1 0448C400     MOV EAX,DWORD PTR DS:[C44804]
00C3FE39  8B00            MOV EAX,DWORD PTR DS:[EAX]
00C3FE3B  E8 ACD1FEFF     CALL 00C2CFEC
00C3FE40  A1 0448C400     MOV EAX,DWORD PTR DS:[C44804]
00C3FE45  8B00            MOV EAX,DWORD PTR DS:[EAX]
00C3FE47  E8 90C7FEFF     CALL 00C2C5DC
00C3FE4C  50              PUSH EAX
00C3FE4D  E8 36010000     CALL 00C3FF88
00C3FE52  56              PUSH ESI
```

Esto ya se ve más clarito está todo desencriptado, traceamos un poco hasta encontrar algún salto que nos pueda ayudar ;-) y lo encontramos:

```
00C2C5E3  C60424 00       MOV BYTE PTR SS:[ESP],0
00C2C5E7  807D 09 00      CMP BYTE PTR SS:[EBP+9],0
00C2C5EB  0F84 92000000   JE 00C2C683
00C2C5F1  8B45 0C         MOV EAX,DWORD PTR SS:[EBP+C]
00C2C5F4  8BF0            MOV ESI,EAX
```

Esa variable [EBP-9] parece tener mucho que ver en el tema del Time-Trial lo primero que haremos será ver qué valor esconde, hacemos un "Follow in dump->Memory address" para ver lo que vale:

```
Address   Hex dump                                          ASCII
00C733C9  00 00 00 31 58 C4 00 00 00 00 00 00 00 00 00 00  ...1X-..........
00C733D9  00 00 00 FF FF FF FF F0 33 C7 00 20 34 C7 00 50  ...    -3Ä. 4Ä.P
00C733E9  34 C7 00 32 00 00 00 C8 54 C2 00 00 00 00 00 00  4Ä.2...╚T┬......
00C733F9  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
00C73409  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
```

Representa [C733C9] y su byte bajo está puesto a 0, se lo cambiamos ;-)

```
Address   Hex dump                                          ASCII
00C733C9  01 00 00 31 58 C4 00 00 00 00 00 00 00 00 00 00  ☺..1X-..........
00C733D9  00 00 00 FF FF FF FF F0 33 C7 00 20 34 C7 00 50  ...    -3Ä. 4Ä.P
00C733E9  34 C7 00 32 00 00 00 C8 54 C2 00 00 00 00 00 00  4Ä.2...╚T┬......
00C733F9  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
00C73409  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
```

Ese 1 nos va a ayudar, continúa con la comparación entre la variable y 0 en la línea:

```
00C2C5E7    807D 09 00      CMP BYTE PTR SS:[EBP+9],0
```

que es donde estamos y llegamos al JE siguiente, como hemos modificado el valor de dicha variable ahora se comportará de otra manera y no va a saltar:

```
00C2C5E3  C60424 00       MOV BYTE PTR SS:[ESP],0
00C2C5E7  807D 09 00      CMP BYTE PTR SS:[EBP+9],0
00C2C5EB  0F84 92000000   JE 00C2C683
00C2C5F1  8B45 0C         MOV EAX,DWORD PTR SS:[EBP+C]
00C2C5F4  8BF0            MOV ESI,EAX
00C2C5F6  0FB63E          MOVZX EDI,BYTE PTR DS:[ESI]
```

Podemos pulsar F9 tranquilamente y e programa corre perfecto ;-)

Y hasta aquí hemos llegado, espero haberos entretenido un rato con eso me conformo, esto surgió de una tarde de aburrimiento en la que me entretuve estudiando un poco lo que hace este packer, no he profundizado más porque tampoco he podido ojalá pudiera hacerlo, además hay mil formas de hacer el trabajo de la IAT con scripts y/o injertos que nos audarían más seuro que hacerlo a mano como yo lo he hecho, tendré que aprender más, todo se andará ;-)

Un saludo a todo CracksLatinoS sin excepción, no quiero nombrar a nadie para no olvidarme de ninguno, saludos a todos y hasta la próxima, cualquire cosa ya sabéis:

syxe00@yahoo.es